

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

KOMAL GUPTA (1BF24CS144)

**in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING**



**B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026**

CERTIFICATE

**B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering**



This is to certify that the Lab work entitled “**ANALYSIS AND DESIGN OF ALGORITHMS**” carried out by KOMAL GUPTA (1BF24CS144), who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - **(23CS4PCADA)** work prescribed for the said degree.

Manjula S
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1	Write program to obtain the Topological ordering of vertices in a given digraph.	4-5
2	Implement Johnson Trotter algorithm to generate permutations.	6-8
3	Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.	9-11
4	Sort a given set of N integer elements using Quick Sort technique and compute its time taken.	12-14
5	Sort a given set of N integer elements using Heap Sort technique and compute its time taken.	15-17
6	Implement 0/1 Knapsack problem using dynamic programming.	18-20
7	Implement All Pair Shortest paths problem using Floyd's algorithm.	21-23
8	(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm. (b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.	24-28
9	Implement Fractional Knapsack using Greedy technique.	29-30
10	From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.	31-33
11	Implement "N-Queens Problem" using Backtracking.	34-35

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

Lab program 1: Write program to obtain the Topological ordering of vertices in a given digraph.

```
#include <stdio.h>

#define MAX 10

int main() {
    int n, i, j;
    int indegree[MAX] = {0};
    int graph[MAX][MAX];
    int order[MAX], count = 0;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter adjacency matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < n; j++) {
            scanf("%d", &graph[i][j]);
            if(graph[i][j] == 1) {
                indegree[j]++;
            }
        }
    }
    while(count < n) {
        int found = 0;
        for(i = 0; i < n; i++) {
            if(indegree[i] == 0) {
                order[count++] = i;
                indegree[i] = -1;
                for(j = 0; j < n; j++) {
                    if(graph[i][j] == 1) {
                        indegree[j]--;
                    }
                }
            }
        }
    }
}
```

```

    }

    found = 1;

    }

    }

    if(!found) break;

}

if(count < n) {

    printf("No Topological Sequence (cycle detected)\n");

} else {

    printf("Topological Order: ");

    for(i = 0; i < n; i++) {

        printf("%d ", order[i]);

    }

    printf("\n");

}

return 0;

}

```

Screenshot of the output:

```

Enter number of vertices: 6
Enter adjacency matrix:
0 0 0 0 0 0
0 0 0 0 0 0
0 0 0 0 0 0
0 0 0 1 0 0
0 1 0 0 0 0
1 1 0 0 0 0
1 0 1 0 0 0
Topological Order: 4 5 0 2 3 1

Process returned 0 (0x0)   execution time : 58.814 s
Press any key to continue.
|

```

Lab program 2: Implement Johnson Trotter algorithm to generate permutations.

```
#include <stdio.h>

#define LEFT 0
#define RIGHT 1

void print(int a[], int dir[], int n) {
    for(int i = 0; i < n; i++) {
        if(dir[a[i]] == LEFT)
            printf("%d<- ", a[i]);
        else
            printf("%d-> ", a[i]);
    }
    printf("\n");
}

int getMobile(int a[], int dir[], int n) {
    int mobile = 0;
    for(int i = 0; i < n; i++) {
        if(dir[a[i]] == LEFT && i > 0) {
            if(a[i] > a[i-1] && a[i] > mobile)
                mobile = a[i];
        }
        if(dir[a[i]] == RIGHT && i < n-1) {
            if(a[i] > a[i+1] && a[i] > mobile)
                mobile = a[i];
        }
    }
    return mobile;
}

int findPos(int a[], int n, int mobile) {
    for(int i = 0; i < n; i++) {
```

```

        if(a[i] == mobile)
            return i;
    }
    return -1;
}

void johnsonTrotter(int n) {
    int a[n], dir[n+1];
    for(int i = 0; i < n; i++) {
        a[i] = i + 1;
        dir[i+1] = LEFT;
    }
    print(a, dir, n);
    while(1) {
        int mobile = getMobile(a, dir, n);
        if(mobile == 0)
            break;
        int pos = findPos(a, n, mobile);
        if(dir[mobile] == LEFT) {
            int temp = a[pos];
            a[pos] = a[pos-1];
            a[pos-1] = temp;
            pos = pos - 1;
        } else {
            int temp = a[pos];
            a[pos] = a[pos+1];
            a[pos+1] = temp;
            pos = pos + 1;
        }
        for(int i = 1; i <= n; i++) {

```

```

        if(i > mobile) {
            dir[i] = !dir[i];
        }
    }

    print(a, dir, n);
}

int main() {
    int n;

    printf("Enter n: ");
    scanf("%d", &n);
    johnsonTrotter(n);
    return 0;
}

```

Screenshot of the output:

```

Enter n: 3
1<- 2<- 3<-
1<- 3<- 2<-
3<- 1<- 2<-
3-> 2<- 1<-
2<- 3-> 1<-
2<- 1<- 3->

Process returned 0 (0x0)   execution time : 2.854 s
Press any key to continue.

```

Lab program 3: Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.

```
#include <stdio.h>

#include <stdlib.h>

#include <time.h>

void merge(int arr[], int lb, int mid, int ub) {
    int n1 = mid - lb + 1;
    int n2 = ub - mid;
    int L[n1], R[n2];
    for (int i = 0; i < n1; i++)
        L[i] = arr[lb + i];
    for (int j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];
    int i = 0, j = 0, k = lb;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j])
            arr[k++] = L[i++];
        else
            arr[k++] = R[j++];
    }
    while (i < n1)
        arr[k++] = L[i++];
    while (j < n2)
        arr[k++] = R[j++];
}

void mergeSort(int arr[], int lb, int ub) {
    if (lb < ub) {
        int mid = (lb + ub) / 2;
```

```

        mergeSort(arr, lb, mid);
        mergeSort(arr, mid + 1, ub);
        merge(arr, lb, mid, ub);
    }
}

int main() {
    int n;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter elements:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    clock_t start = clock();
    mergeSort(arr, 0, n - 1);
    clock_t end = clock();

    double time_taken =
        (double)(end - start) / CLOCKS_PER_SEC;

    printf("\nSorted array:\n");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    printf("\n");

    printf("Time taken: %.6f seconds\n", time_taken);

    return 0;
}

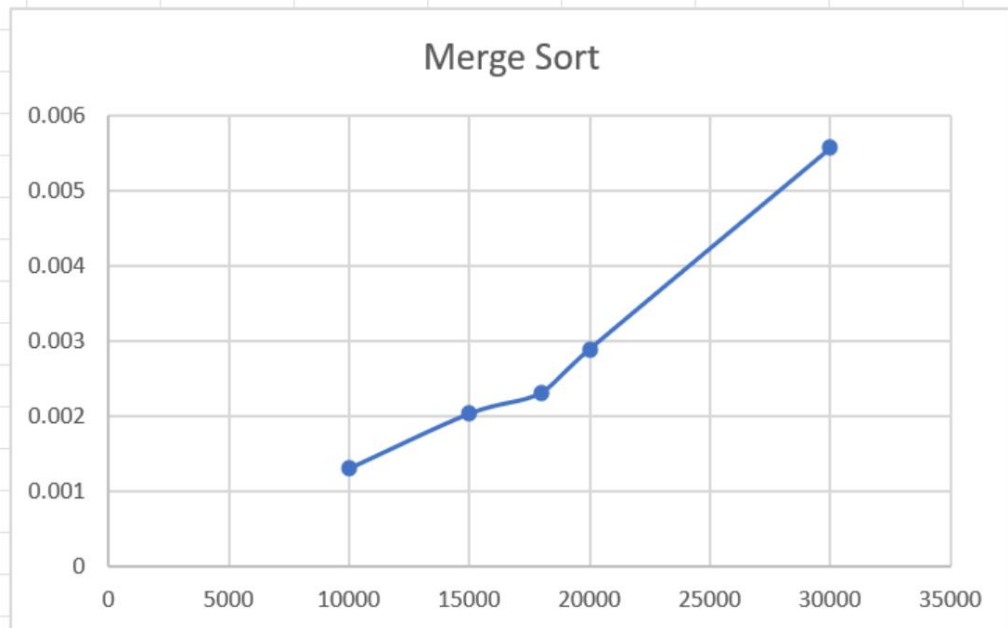
```

Screenshot of the output:

```
Enter number of elements: 5
Enter elements:
29 5 67 3 54

Sorted array:
3 5 29 54 67
Time taken: 0.000000 seconds

Process returned 0 (0x0)   execution time : 10.999 s
Press any key to continue.
```



Lab program 4: Sort a given set of N integer elements using Quick Sort technique and compute its time taken.

```
#include <stdio.h>

#include <stdlib.h>

#include <time.h>

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int partition(int arr[], int low, int high) {
    int pivot = arr[high]; // choosing last element as pivot
    int i = (low - 1);
    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return (i + 1);
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);

        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    int N;

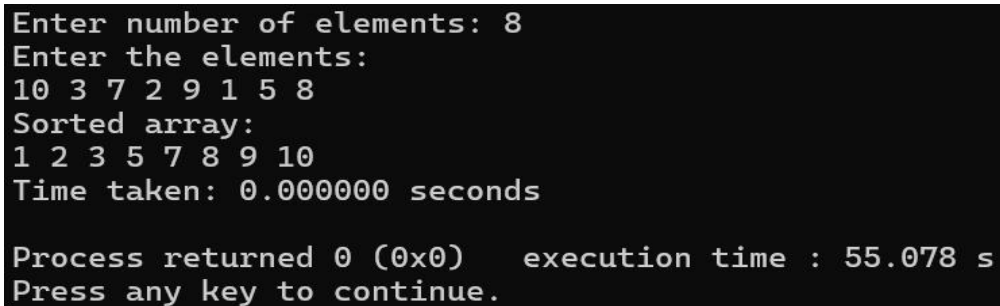
    printf("Enter number of elements: ");
```

```

scanf("%d", &N);
int arr[N];
printf("Enter the elements:\n");
for (int i = 0; i < N; i++) {
    scanf("%d", &arr[i]);
}
clock_t start, end;
start = clock();
quickSort(arr, 0, N - 1);
end = clock();
printf("Sorted array:\n");
for (int i = 0; i < N; i++) {
    printf("%d ", arr[i]);
}
double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
printf("\nTime taken: %f seconds\n", time_taken);
return 0;
}

```

Screenshot of the output :

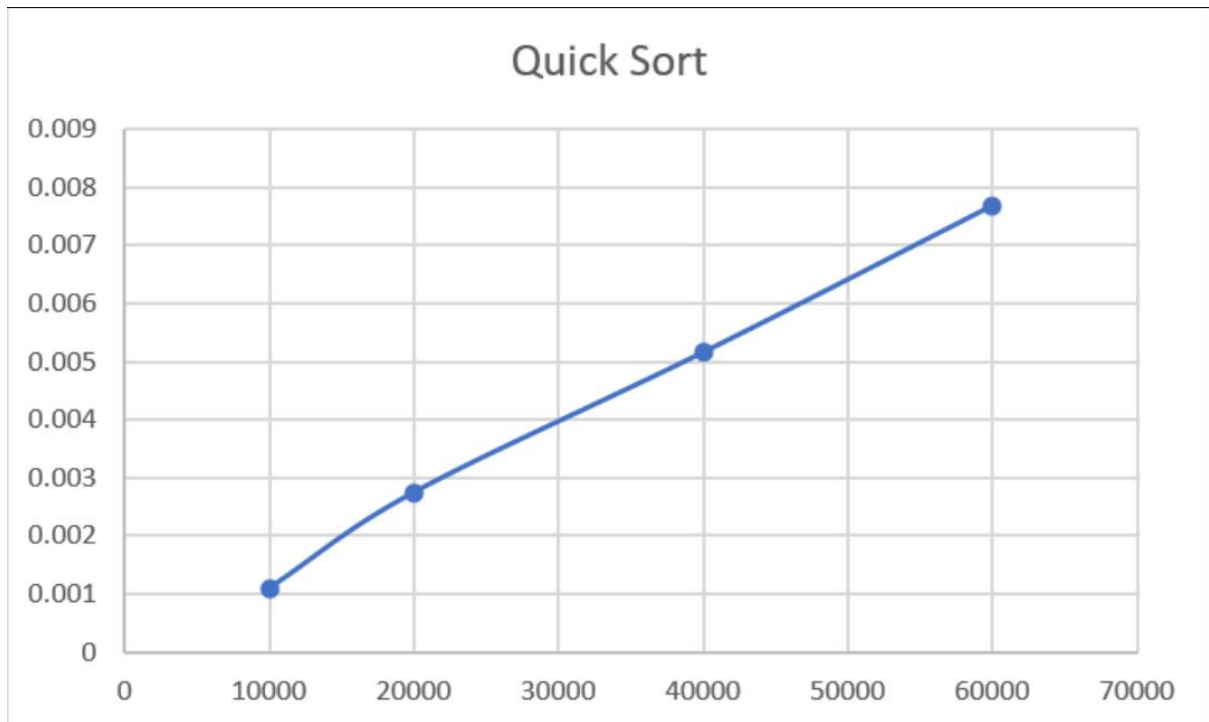


```

Enter number of elements: 8
Enter the elements:
10 3 7 2 9 1 5 8
Sorted array:
1 2 3 5 7 8 9 10
Time taken: 0.000000 seconds

Process returned 0 (0x0)    execution time : 55.078 s
Press any key to continue.
|

```



Lab program 5: Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

```
#include <stdio.h>

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

void siftUp(int a[], int i) {
    while(i > 0) {
        int parent = (i - 1) / 2;
        if(a[parent] < a[i]) {
            swap(&a[parent], &a[i]);
            i = parent;
        } else {
            break;
        }
    }
}

void buildHeap(int a[], int n) {
    for(int i = 1; i < n; i++) {
        siftUp(a, i);
    }
}

void heapify(int a[], int n, int i) {
    int largest = i;
    int left = 2*i + 1;
    int right = 2*i + 2;

    if(left < n && a[left] > a[largest])
```

```

        largest = left;
    if(right < n && a[right] > a[largest])
        largest = right;
    if(largest != i) {
        swap(&a[i], &a[largest]);
        heapify(a, n, largest);
    }
}

void heapSort(int a[], int n) {
    buildHeap(a, n);
    for(int i = n-1; i > 0; i--) {
        swap(&a[0], &a[i]);
        heapify(a, i, 0);
    }
}

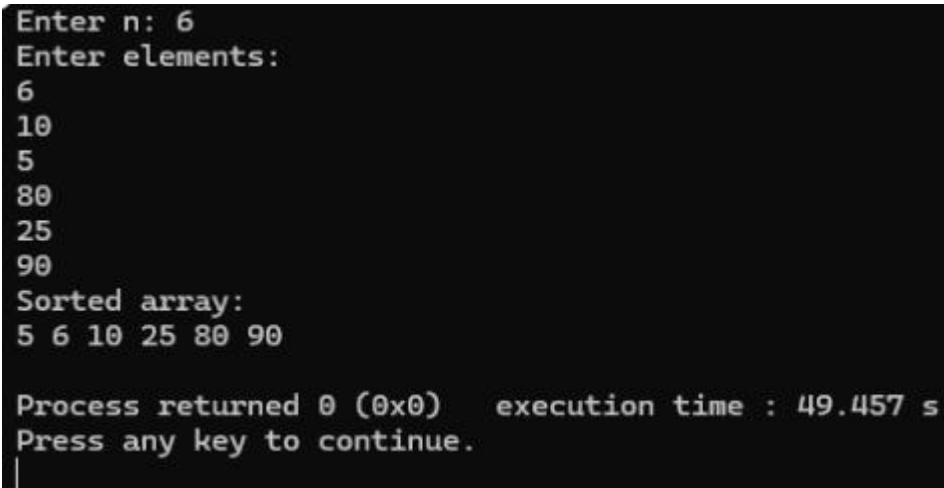
void print(int a[], int n) {
    for(int i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
}

int main() {
    int n;
    printf("Enter n: ");
    scanf("%d", &n);
    int a[n];
    printf("Enter elements:\n");
    for(int i = 0; i < n; i++)
        scanf("%d", &a[i]);
    heapSort(a, n);
}

```

```
    printf("Sorted array:\n");  
    print(a, n);  
    return 0;  
}
```

Screenshot of the output:



```
Enter n: 6  
Enter elements:  
6  
10  
5  
80  
25  
90  
Sorted array:  
5 6 10 25 80 90  
  
Process returned 0 (0x0)   execution time : 49.457 s  
Press any key to continue.  
|
```

Lab program 6: Implement 0/1 Knapsack problem using dynamic programming.

```
#include <stdio.h>

int knapsack(int W, int wt[], int val[], int n) {
    int dp[n+1][W+1];
    for (int i = 0; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            if (i == 0 || w == 0) {
                dp[i][w] = 0;
            }
            else if (wt[i-1] <= w) {
                int include = val[i-1] + dp[i-1][w - wt[i-1]];
                int exclude = dp[i-1][w];
                dp[i][w] = (include > exclude) ? include : exclude;
            }
            else {
                dp[i][w] = dp[i-1][w];
            }
        }
    }
    int selected[n];
    for (int i = 0; i < n; i++)
        selected[i] = 0;
    int i = n;
    int w = W;
    while (i > 0 && w > 0) {
        if (dp[i][w] != dp[i-1][w]) {
            selected[i-1] = 1;
            w = w - wt[i-1];
        }
    }
}
```

```

    }
    i--;
}

printf("\nItems included in Knapsack:\n");

for (int i = 0; i < n; i++) {
    printf("%d ", selected[i]);
}
printf("\n");
return dp[n][W];
}

int main() {
    int n, W;
    printf("Enter number of items: ");
    scanf("%d", &n);
    int val[n], wt[n];
    printf("Enter profit of items:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &val[i]);
    }
    printf("Enter weights of items:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &wt[i]);
    }
    printf("Enter knapsack capacity: ");
    scanf("%d", &W);
    int result = knapsack(W, wt, val, n);
    printf("\nMaximum value in Knapsack = %d\n", result);
}

```

```
    return 0;  
}
```

Screenshot of the output:

```
Enter number of items: 5  
Enter profit of items:  
25 20 15 40 50  
Enter weights of items:  
3 2 1 4 5  
Enter knapsack capacity: 6  
  
Items included in Knapsack:  
0 0 1 0 1  
  
Maximum value in Knapsack = 65  
  
Process returned 0 (0x0)   execution time : 23.550 s  
Press any key to continue.  
|
```

Lab program 7: Implement All Pair Shortest paths problem using Floyd's algorithm.

```
#include <stdio.h>

#define INF 99999
#define MAX 100

void printSolution(int dist[MAX][MAX], int V) {
    printf("\nShortest distance matrix:\n");
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            if (dist[i][j] == INF)
                printf("%7s", "INF");
            else
                printf("%7d", dist[i][j]);
        }
        printf("\n");
    }
}

void floydWarshall(int graph[MAX][MAX], int V) {
    int dist[MAX][MAX];

    for (int i = 0; i < V; i++)
        for (int j = 0; j < V; j++)
            dist[i][j] = graph[i][j];

    // Floyd-Warshall
    for (int k = 0; k < V; k++) {
        for (int i = 0; i < V; i++) {
            for (int j = 0; j < V; j++) {
                if (dist[i][k] < INF && dist[k][j] < INF &&
                    dist[i][k] + dist[k][j] < dist[i][j]) {
                    dist[i][j] = dist[i][k] + dist[k][j];
                }
            }
        }
    }
}
```

```

        }
    }
    for (int i = 0; i < V; i++) {
        if (dist[i][i] < 0) {
            printf("\nNegative cycle detected!\n");
            return;
        }
    }
    printSolution(dist, V);
}

int main() {
    int V;
    int graph[MAX][MAX];
    printf("Enter number of vertices: ");
    scanf("%d", &V);
    printf("\nEnter adjacency matrix (-1 for no edge):\n");
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            scanf("%d", &graph[i][j]);
            if (graph[i][j] == -1)
                graph[i][j] = INF;
        }
    }
    for (int i = 0; i < V; i++)
        graph[i][i] = 0;
    floydWarshall(graph, V);
    return 0;}

```

Screenshot of the output:

```
Enter number of vertices: 4

Enter adjacency matrix (-1 for no edge):
0 3 -1 5
2 0 -1 4
-1 6 0 -1
-1 -1 2 0

Shortest distance matrix:
    0    3    7    5
    2    0    6    4
    8    6    0   10
   10    8    2    0

Process returned 0 (0x0)    execution time : 30.376 s
Press any key to continue.
```

Lab program 8a: Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.

```
#include <stdio.h>

#include <limits.h>

int minKey(int key[], int mstSet[], int V) {
    int min = INT_MAX, min_index = -1;
    for (int v = 0; v < V; v++)
        if (mstSet[v] == 0 && key[v] < min)
            min = key[v], min_index = v;
    return min_index;
}

void primMST(int graph[50][50], int V) {
    int parent[V], key[V], mstSet[V];
    int totalCost = 0;
    for (int i = 0; i < V; i++) {
        key[i] = INT_MAX;
        mstSet[i] = 0;
    }
    key[0] = 0;
    parent[0] = -1;
    for (int count = 0; count < V - 1; count++) {
        int u = minKey(key, mstSet, V);
        mstSet[u] = 1;
        for (int v = 0; v < V; v++)
            if (graph[u][v] && mstSet[v] == 0 && graph[u][v] < key[v]) {
                parent[v] = u;
                key[v] = graph[u][v];
            }
    }
}
```

```

printf("\nEdges in MST:\n");
for (int i = 1; i < V; i++) {
    printf("%d - %d : %d\n", parent[i], i, graph[i][parent[i]]);
    totalCost += graph[i][parent[i]];
}
printf("Total cost = %d\n", totalCost);
}

int main() {
    int V;
    int graph[50][50];
    printf("Enter number of vertices: ");
    scanf("%d", &V);
    printf("Enter adjacency matrix (enter 0 if no edge, otherwise enter cost):\n");
    for (int i = 0; i < V; i++)
        for (int j = 0; j < V; j++)
            scanf("%d", &graph[i][j]);
    primMST(graph, V);
    return 0;
}

```

Screenshot of the output:

```

Enter number of vertices: 4
Enter adjacency matrix (enter 0 if no edge, otherwise enter cost):
0 0 0 1
0 0 1 0
0 1 0 1
1 0 1 0

Edges in MST:
0 - 3 : 1
3 - 2 : 1
2 - 1 : 1
Total cost = 3

Process returned 0 (0x0)   execution time : 21.271 s
Press any key to continue.
|

```

Lab program 8b: Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.

```
#include <stdio.h>

#include <stdlib.h>

struct Edge {
    int src,dest,weight;
};

struct Subset {
    int parent;
};

int find(struct Subset subsets[],int i){
    while(subsets[i].parent!=i)
        i=subsets[i].parent;
    return i;
}

void Union(struct Subset subsets[],int x,int y){
    int xroot=find(subsets,x);
    int yroot=find(subsets,y);
    subsets[xroot].parent=yroot;
}

int compare(const void *a,const void *b){
    return ((struct Edge*)a)->weight-((struct Edge*)b)->weight;
}

void KruskalMST(struct Edge edges[],int V,int E){
    struct Edge result[V];
    struct Subset subsets[V];
    qsort(edges,E,sizeof(struct Edge),compare);
    for(int i=0;i<V;i++)
        subsets[i].parent=i;
```

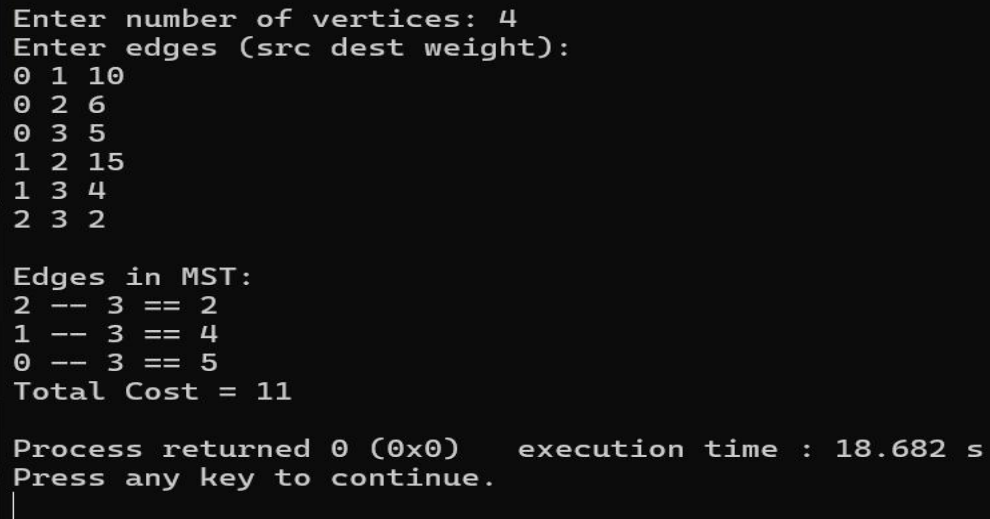
```

int e=0;
int i=0;
while(e<V-1&& i<E){
    struct Edge next=edges[i++];
    int x=find(subsets,next.src);
    int y=find(subsets,next.dest);
    if(x!=y){
        result[e++]=next;
        Union(subsets,x,y);
    }
}
printf("\nEdges in MST:\n");
int total=0;
for(i=0;i<e;i++){
    printf("%d -- %d == %d\n",result[i].src,result[i].dest,result[i].weight);
    total+=result[i].weight;
}
printf("Total Cost = %d\n",total);
}
int main(){
    int V;
    printf("Enter number of vertices: ");
    scanf("%d",&V);
    int E=V*(V-1)/2;
    struct Edge edges[E];
    printf("Enter edges (src dest weight):\n");
    for(int i=0;i<E;i++){
        scanf("%d %d %d",&edges[i].src,&edges[i].dest,&edges[i].weight);
    }
}

```

```
KruskalMST(edges,V,E);  
return 0;  
}
```

Screenshot of the output:



```
Enter number of vertices: 4  
Enter edges (src dest weight):  
0 1 10  
0 2 6  
0 3 5  
1 2 15  
1 3 4  
2 3 2  
  
Edges in MST:  
2 -- 3 == 2  
1 -- 3 == 4  
0 -- 3 == 5  
Total Cost = 11  
  
Process returned 0 (0x0)   execution time : 18.682 s  
Press any key to continue.  
|
```

Lab program 9: Implement Fractional Knapsack using Greedy technique

```
#include <stdio.h>

struct Item{int weight;int value;};

int main(){
    int n,capacity;

    printf("Enter number of items: ");
    scanf("%d",&n);

    struct Item items[n];

    for(int i=0;i<n;i++){
        printf("Enter weight and value of item %d: ",i+1);
        scanf("%d %d",&items[i].weight,&items[i].value);
    }

    printf("Enter knapsack capacity: ");
    scanf("%d",&capacity);

    double ratio[n];

    for(int i=0;i<n;i++)ratio[i]=(double)items[i].value/items[i].weight;
    for(int i=0;i<n-1;i++){
        for(int j=i+1;j<n;j++){
            if(ratio[i]<ratio[j]){
                double tempR=ratio[i];ratio[i]=ratio[j];ratio[j]=tempR;
                struct Item temp=items[i];items[i]=items[j];items[j]=temp;
            }
        }
    }

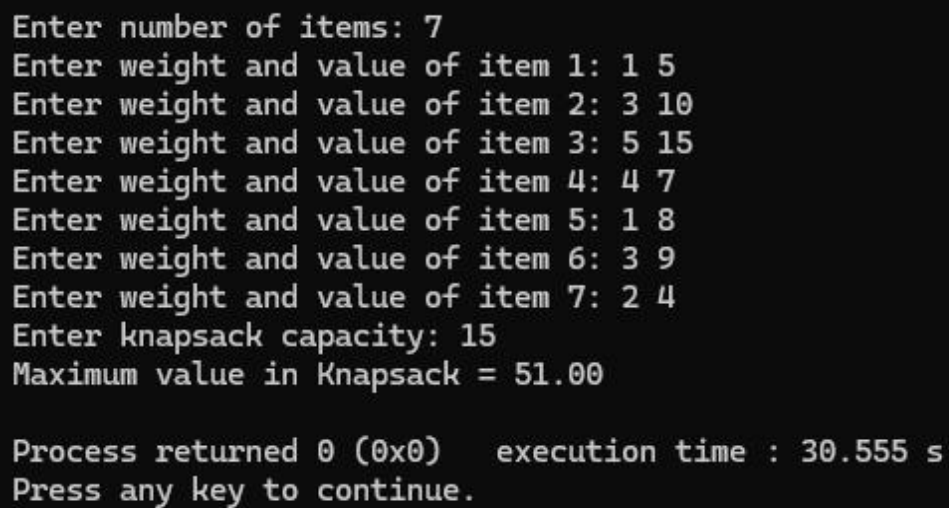
    double maxValue=0.0;int remaining=capacity;
    for(int i=0;i<n;i++){
        if(items[i].weight<=remaining){
            remaining-=items[i].weight;
```

```
        maxValue+=items[i].value;
    }else{
        maxValue+=ratio[i]*remaining;
        break;
    }
}

printf("Maximum value in Knapsack = %.2f\n",maxValue);

return 0;
}
```

Screenshot of the output:



```
Enter number of items: 7
Enter weight and value of item 1: 1 5
Enter weight and value of item 2: 3 10
Enter weight and value of item 3: 5 15
Enter weight and value of item 4: 4 7
Enter weight and value of item 5: 1 8
Enter weight and value of item 6: 3 9
Enter weight and value of item 7: 2 4
Enter knapsack capacity: 15
Maximum value in Knapsack = 51.00

Process returned 0 (0x0)   execution time : 30.555 s
Press any key to continue.
|
```

Lab program 10: From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

```
#include <stdio.h>

#include <limits.h>

#define V 10 // maximum number of vertices

int minDistance(int dist[], int sptSet[], int vertices) {
    int min = INT_MAX, min_index;

    for (int v = 0; v < vertices; v++) {
        if (sptSet[v] == 0 && dist[v] <= min) {
            min = dist[v];
            min_index = v;
        }
    }

    return min_index;
}

void printSolution(int dist[], int vertices, int src) {
    printf("Vertex \t Distance from Source %d\n", src);
    for (int i = 0; i < vertices; i++) {
        printf("%d \t %d\n", i, dist[i]);
    }
}

void dijkstra(int graph[V][V], int src, int vertices) {
    int dist[vertices]; // shortest distance from src
    int sptSet[vertices]; // whether vertex is included in shortest path tree

    for (int i = 0; i < vertices; i++) {
        dist[i] = INT_MAX;
        sptSet[i] = 0;
    }
}
```

```

    dist[src] = 0;
    for (int count = 0; count < vertices - 1; count++) {
        int u = minDistance(dist, sptSet, vertices);
        sptSet[u] = 1;
        for (int v = 0; v < vertices; v++) {
            if (!sptSet[v] && graph[u][v] && dist[u] != INT_MAX
                && dist[u] + graph[u][v] < dist[v]) {
                dist[v] = dist[u] + graph[u][v];
            }
        }
    }
    printSolution(dist, vertices, src);
}

int main() {
    int vertices;

    printf("Enter number of vertices: ");
    scanf("%d", &vertices)

    int graph[V][V];

    printf("Enter adjacency matrix (0 if no edge):\n");
    for (int i = 0; i < vertices; i++) {
        for (int j = 0; j < vertices; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    int src;

    printf("Enter source vertex: ");
    scanf("%d", &src);

```

```
dijkstra(graph, src, vertices);

return 0;
}
```

Screenshot of the output:

```
Enter number of vertices: 5
Enter adjacency matrix (0 if no edge):
0 10 0 5 0
0 0 1 2 0
0 0 0 0 4
0 3 9 0 2
7 0 6 0 0
Enter source vertex: 0
Vertex    Distance from Source 0
0         0
1         8
2         9
3         5
4         7

Process returned 0 (0x0)    execution time : 21.083 s
Press any key to continue.
|
```

Lab program 11: Implement “N-Queens Problem” using Backtracking.

```
#include <stdio.h>

#include <stdbool.h>

#define MAX 20

int N;

int board[MAX][MAX];

bool isSafe(int row,int col){

    for(int i=0;i<row;i++){

        if(board[i][col])return false;

        if(col-(row-i)>=0&&board[i][col-(row-i)])return false;

        if(col+(row-i)<N&&board[i][col+(row-i)])return false;

    }

    return true;

}

void printBoard(){

    for(int i=0;i<N;i++){

        for(int j=0;j<N;j++)printf("%c ",board[i][j]?'Q':'.');

        printf("\n");

    }

    printf("\n");

}

void solve(int row){

    if(row==N){

        printBoard();

        return;

    }

    for(int col=0;col<N;col++){

        if(isSafe(row,col)){

            board[row][col]=1;
```

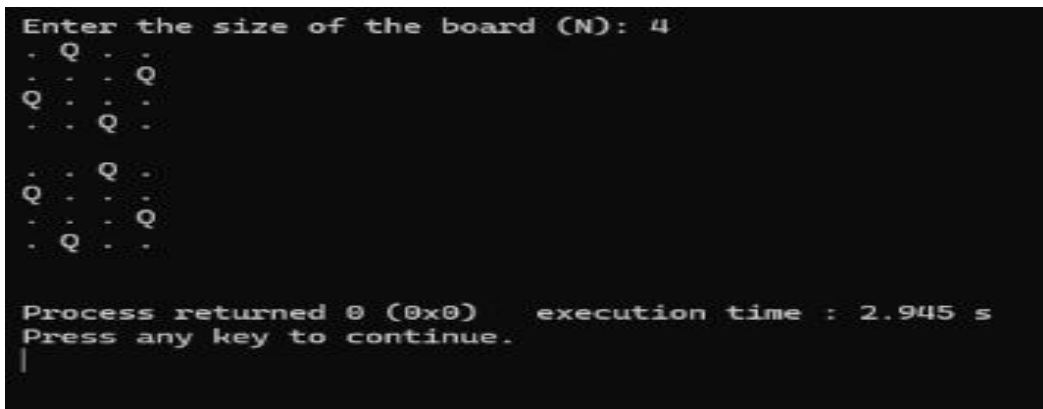
```

        solve(row+1);
        board[row][col]=0;
    }
}
}

int main(){
    printf("Enter the size of the board (N): ");
    scanf("%d",&N);
    if(N>MAX||N<1){
        printf("Invalid board size. Please enter between 1 and %d.\n",MAX);
        return 1;
    }
    solve(0);
    return 0;
}

```

Screenshot of the output:



```

Enter the size of the board (N): 4
. Q . .
. . . Q
Q . . .
. . Q .

. . Q .
Q . . .
. . . Q
. Q . .

Process returned 0 (0x0)   execution time : 2.945 s
Press any key to continue.
|

```

LEETCODES

Leetcode 3658: GCD of odd and even sums

```
int gcdOfOddEvenSums(int n) {  
    int oddSum = n * n;  
    int evenSum = n * (n + 1);  
    // find gcd using Euclidean algorithm  
    while (evenSum != 0) {  
        int temp = evenSum;  
        evenSum = oddSum % evenSum;  
        oddSum = temp;  
    }  
    return oddSum;  
}
```



Leetcode 2447: No of subarrays with GCD equal to k

```
int gcd(int a, int b) {  
    while (b != 0) {  
        int temp = b;  
        b = a % b;  
        a = temp;  
    }  
    return a;  
}  
  
int subarrayGCD(int* nums, int numsSize, int k) {  
  
    int count = 0;  
  
    for (int i = 0; i < numsSize; i++) {  
  
        int currentGCD = 0;  
  
        for (int j = i; j < numsSize; j++) {  
  
            currentGCD = gcd(currentGCD, nums[j]);  
  
            if (currentGCD == k) {  
                count++;  
            }  
            else if (currentGCD < k) {  
                break;  
            }  
        }  
    }  
}
```

```
    }  
}  
  
return count;  
}
```

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2

Input

nums =
[9,3,1,2,6,3]



k =
3

Output

4

Expected

4

Leetcode 1819: No of different subsequences GCDs

```
int gcd(int a, int b) {  
    while (b != 0) {  
        int temp = b;  
        b = a % b;  
        a = temp;  
    }  
    return a;  
}  
  
int countDifferentSubsequenceGCDs(int* nums, int numsSize) {  
  
    int maxNum = 0;  
  
    for (int i = 0; i < numsSize; i++) {  
        if (nums[i] > maxNum) {  
            maxNum = nums[i];  
        }  
    }  
  
    int* exists = (int*)calloc(maxNum + 1, sizeof(int));  
  
    for (int i = 0; i < numsSize; i++) {  
        exists[nums[i]] = 1;  
    }  
  
    int answer = 0;  
  
    for (int x = 1; x <= maxNum; x++) {
```

```

    int currentGCD = 0;

    for (int multiple = x; multiple <= maxNum; multiple += x) {

        if (exists[multiple]) {

            currentGCD = gcd(currentGCD, multiple);

            if (currentGCD == x) {

                answer++;

                break;

            }

        }

    }

}

free(exists);

return answer;

}

```

Accepted Runtime: 0 ms

☒ Case 1
 ☒ Case 2

Input

nums =
[6,10,3]

Output

5

Expected

5

Leetcode 3867: Sum of GCD of formed pairs

```
int gcd(int a, int b) {
    while (b != 0) {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}

int cmp(const void* a, const void* b) {
    return (*(int*)a - *(int*)b);
}

long long gcdSum(int* nums, int numsSize) {

    int* prefixGcd = (int*)malloc(numsSize * sizeof(int));

    int mx = 0;

    // build prefixGcd array
    for (int i = 0; i < numsSize; i++) {

        if (nums[i] > mx) {
            mx = nums[i];
        }

        prefixGcd[i] = gcd(nums[i], mx);
    }

    qsort(prefixGcd, numsSize, sizeof(int), cmp);
```

```
long long sum = 0;

int left = 0;
int right = numsSize - 1;
while (left < right) {

    sum += gcd(prefixGcd[left], prefixGcd[right]);

    left++;
    right--;
}

free(prefixGcd);
return sum;
}
```

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2

Input
nums =
[2,6,4]

Output
2

Expected
2

Leetcode 207: Course Schedule

```
bool canFinish(int numCourses, int** prerequisites, int prerequisitesSize, int* prerequisitesColSize) {
```

```
    int* indegree = (int*)calloc(numCourses, sizeof(int));
```

```
    int** graph = (int**)malloc(numCourses * sizeof(int*));
```

```
    int* graphSize = (int*)calloc(numCourses, sizeof(int));
```

```
    for (int i = 0; i < numCourses; i++) {
```

```
        graph[i] = (int*)malloc(prerequisitesSize * sizeof(int));
```

```
    }
```

```
    // build graph
```

```
    for (int i = 0; i < prerequisitesSize; i++) {
```

```
        int a = prerequisites[i][0];
```

```
        int b = prerequisites[i][1];
```

```
        graph[b][graphSize[b]++] = a;
```

```
        indegree[a]++;
```

```
    }
```

```
    int* queue = (int*)malloc(numCourses * sizeof(int));
```

```
    int front = 0, rear = 0;
```

```
    // push nodes with indegree 0
```

```
    for (int i = 0; i < numCourses; i++) {
```

```
        if (indegree[i] == 0) {
```

```
            queue[rear++] = i;
```

```

    }
}

int count = 0;

while (front < rear) {

    int node = queue[front++];
    count++;

    for (int i = 0; i < graphSize[node]; i++) {

        int next = graph[node][i];
        indegree[next]--;

        if (indegree[next] == 0) {
            queue[rear++] = next;
        }
    }
}

for (int i = 0; i < numCourses; i++) {
    free(graph[i]);
}

free(graph);
free(graphSize);
free(indegree);
free(queue);

```

```
    return count == numCourses;  
}
```

Accepted

Runtime: 0 ms

✓ Case 1

✓ Case 2

Input

numCourses =
2

prerequisites =
[[1,0]]

Output

true

Expected

true

Leetcode 1334: Find the city with the smallest no of neighbors at a threshold distance

```
#define INF 100000000
```

```
int findTheCity(int n, int** edges, int edgesSize, int* edgesColSize, int distanceThreshold) {
```

```
    int dist[101][101];
```

```
    for (int i = 0; i < n; i++) {
```

```
        for (int j = 0; j < n; j++) {
```

```
            if (i == j)
```

```
                dist[i][j] = 0;
```

```
            else
```

```
                dist[i][j] = INF;
```

```
        }
```

```
    }
```

```
    for (int i = 0; i < edgesSize; i++) {
```

```
        int u = edges[i][0];
```

```
        int v = edges[i][1];
```

```
        int w = edges[i][2];
```

```
        dist[u][v] = w;
```

```
        dist[v][u] = w;
```

```
    }
```

```
    for (int k = 0; k < n; k++) {
```

```

for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {

        if (dist[i][k] + dist[k][j] < dist[i][j]) {
            dist[i][j] = dist[i][k] + dist[k][j];
        }
    }
}

int answer = -1;
int minReachable = INF;

for (int i = 0; i < n; i++) {

    int count = 0;

    for (int j = 0; j < n; j++) {

        if (i != j && dist[i][j] <= distanceThreshold) {
            count++;
        }
    }

    if (count <= minReachable) {
        minReachable = count;
        answer = i;
    }
}

```

```
    return answer;  
}
```

Accepted Runtime: 0 ms

✓ Case 1

✓ Case 2

Input

n =

4

edges =

[[0,1,3],[1,2,1],[1,3,4],[2,3,1]]

distanceThreshold =

4



Output

3

Expected

3

Leetcode 912: Sort an array

```
void merge(int* nums, int left, int mid, int right) {

    int n1 = mid - left + 1;
    int n2 = right - mid;

    int L[n1], R[n2];

    for (int i = 0; i < n1; i++)
        L[i] = nums[left + i];

    for (int i = 0; i < n2; i++)
        R[i] = nums[mid + 1 + i];

    int i = 0, j = 0, k = left;

    while (i < n1 && j < n2) {

        if (L[i] <= R[j]) {
            nums[k++] = L[i++];
        } else {
            nums[k++] = R[j++];
        }
    }

    while (i < n1) {
        nums[k++] = L[i++];
    }
}
```

```

while (j < n2) {
    nums[k++] = R[j++];
}
}

void mergeSort(int* nums, int left, int right) {

    if (left >= right)
        return;

    int mid = left + (right - left) / 2;

    mergeSort(nums, left, mid);
    mergeSort(nums, mid + 1, right);

    merge(nums, left, mid, right);
}

int* sortArray(int* nums, int numsSize, int* returnSize) {

    mergeSort(nums, 0, numsSize - 1);

    *returnSize = numsSize;

    return nums;
}

```

Accepted Runtime: 0 ms

☒ Case 1

☒ Case 2

Input

```
nums =  
[5,2,3,1]
```

Output

```
[1,2,3,5]
```

Expected

```
[1,2,3,5]
```

Leetcode 494: Target Sum

```
int findTargetSumWays(int* nums, int numsSize, int target) {
```

```
    int totalSum = 0;
```

```
    for (int i = 0; i < numsSize; i++) {
```

```
        totalSum += nums[i];
```

```
    }
```

```
    // impossible case
```

```
    if ((target + totalSum) % 2 != 0 || abs(target) > totalSum) {
```

```
        return 0;
```

```
    }
```

```
    int subsetSum = (target + totalSum) / 2;
```

```
    int dp[subsetSum + 1];
```

```
    for (int i = 0; i <= subsetSum; i++) {
```

```
        dp[i] = 0;
```

```
    }
```

```
    dp[0] = 1;
```

```
    for (int i = 0; i < numsSize; i++) {
```

```
        for (int j = subsetSum; j >= nums[i]; j--) {
```

```
            dp[j] += dp[j - nums[i]];
```

```
    }  
}  
  
return dp[subsetSum];  
}
```

Accepted

Runtime: 0 ms

✓ Case 1

✓ Case 2

Input

nums =
[1,1,1,1,1]

target =
3



Output

5

Expected

5

Leetcode 316: Remove duplicate letters

```
char* removeDuplicateLetters(char* s) {

    int freq[26] = {0};
    int visited[26] = {0};

    int n = strlen(s);

    for (int i = 0; i < n; i++) {
        freq[s[i] - 'a']++;
    }

    char* stack = (char*)malloc((n + 1) * sizeof(char));
    int top = -1;

    for (int i = 0; i < n; i++) {

        char ch = s[i];

        freq[ch - 'a']--;

        if (visited[ch - 'a']) {
            continue;
        }

        while (top >= 0 &&
               stack[top] > ch &&
               freq[stack[top] - 'a'] > 0) {
```

```
        visited[stack[top] - 'a'] = 0;

        top--;
    }

    stack[++top] = ch;
    visited[ch - 'a'] = 1;
}

stack[top + 1] = '\0';

return stack;
}
```

Accepted Runtime: 0 ms

✓ Case 1

✓ Case 2

Input

s =
"bcabc"

Output

"abc"

Expected

"abc"